MODERN RECURRENT NEURAL NETWORKS - Report

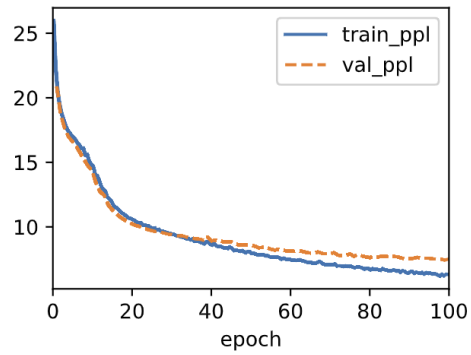
Justin Williams

10909201

Part 1: Basic RNNs

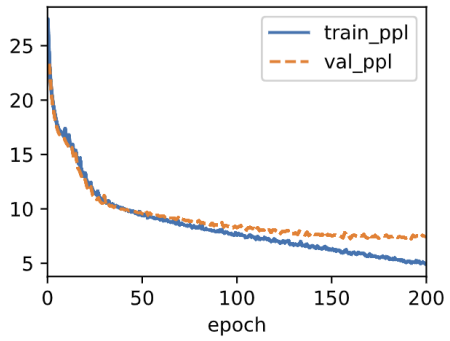
Task 1:

Start & After Optimization:



batch_size 1024
num_steps 32
num_hiddens 32
max_epochs 100

Output: 'it has an an an an an a'



batch_size 2048
num_steps 32
num_hiddens 128
max_epochs 200

Output: 'it has a mone the peas the'

Conclusion:
The goal was to minimize validation perplexity on the Time Machine dataset using only the basic RNN architecture implemented from scratch, tuning four hyperparameters: number of hidden units, number of time steps, batch size, and number of epochs.

The baseline configuration used 32 hidden units, a batch size of 1024, 32 time steps, and 100 epochs, achieving a train perplexity of ~7 and validation perplexity of ~8. While it converged cleanly, the model's limited capacity was evident in its prediction output. It repeatedly predicted the same high frequency token ('an an an an'), showing it lacked the memory to generate meaningful sequences.

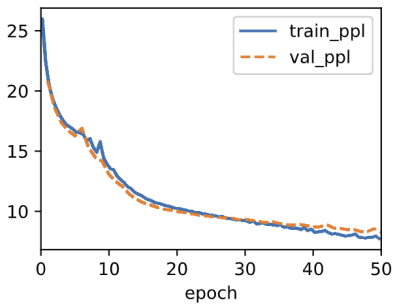
The second and best performing configuration kept num_steps=32 to preserve fast epochs, increased num_hiddens to 128, bumped batch_size to 2048, and extended training to 200 epochs. This allowed the model to fully leverage its extra capacity, pushing train_ppl down to ~5

and val_ppl to ~7.5. The prediction output ('it has a mone the peas the') showed a clear qualitative improvement. It has real words being generated rather than repetitive tokens.

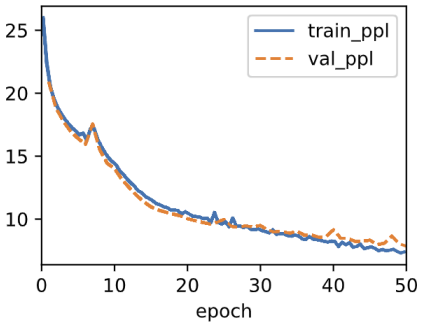
The widening gap between train and validation perplexity in the best configuration signals that the vanilla RNN is approaching its architectural ceiling. Further tuning yields diminishing returns, as the core limitations are inherent to the architecture itself rather than the hyperparameter choices. The best achievable validation perplexity with this architecture was ~7.5, and meaningful further improvement would require upgrading to a GRU or LSTM.

Task 2:

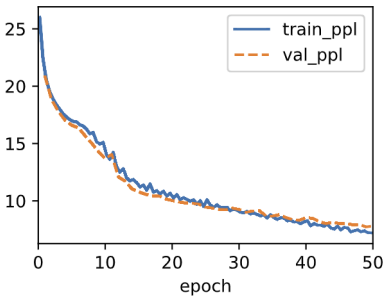
No Gradient Clipping:



ReLU No Clipping:



ReLU With Clipping:



Results:

Config	Final train_ppl	Final val_ppl	Stability
No Clipping	~8	~8	Minor Spike
ReLU No Clipping	~7.5	~8	Big Spike, Most Noise
ReLU With Clipping	~8	~8	Smoothest Curve

Conclusion:

Training the RNN without gradient clipping produced a curve that was mostly well behaved, but showed a visible spike around epoch 7 where perplexity briefly jumped before recovering. This is the exploding gradient problem in action without clipping, a single large gradient update can temporarily destabilize the model. The fact that it recovered is somewhat lucky and not guaranteed, particularly on more complex datasets or with larger models.

Replacing tanh with ReLU and removing clipping produced more instability than the no clipping tanh model. The spike around epoch 10 was more pronounced and the curve was noisier throughout training. This is expected because ReLU is unbounded unlike tanh which squashes outputs to the range (-1, 1), ReLU allows hidden state values to grow without limit across time steps, making exploding gradients more likely and more severe.

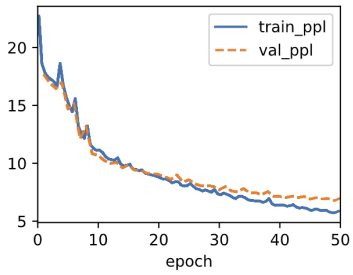
Adding gradient clipping back alongside ReLU produced the smoothest curve of all three configurations, with no visible spikes and steady convergence. This directly answers the question, gradient clipping is still needed with ReLU, and arguably more so than with tanh. The combination of ReLU and clipping stabilizes training effectively, though the final perplexity is similar across all three runs at ~8, suggesting that activation function choice matters more for stability than for final performance at this scale.

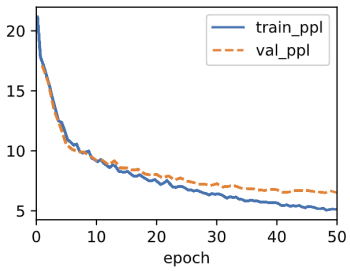
The key takeaway is that gradient clipping is not activation function specific, it is a general safeguard against exploding gradients that benefits any RNN regardless of whether tanh or ReLU is used.

Part 2: GRU

Task 1:

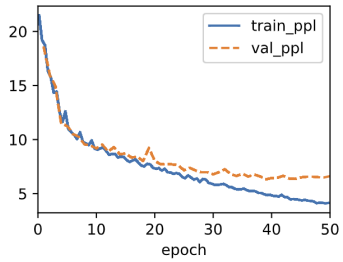
Base Graphs:





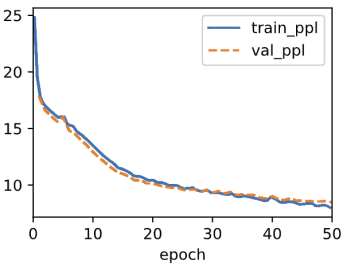
Output:
'it has the time the time t'

New Graphs:



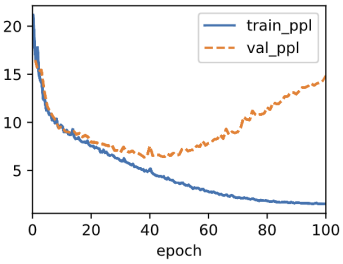
More Hidden Units

Output:
'it has is the filby the fi'



Lower Learning Rate

Output:
'it has and the the the the'



More Hidden Units & More Epochs
Output:
'it has it always been rega'

Results:

Config	num_hiddens	lr	max_epochs	train_ppl	val_ppl	Output Quality
Base GRUScratch	32	4	50	~6	~7	repetitive
Base nn.GRU	32	4	50	~5	~6.5	‘the time the time’
More Hidden	64	4	50	~4	~6.5	‘is the filby the fi’
Lower lr	64	1	50	~8	~8.5	‘And the the the’
Hidden & Epochs	128	4	100	~2	~15	‘It always been rega’

Conclusion:
The base nn.GRU configuration with 32 hidden units and learning rate 4 served as a strong starting point, achieving a val_ppl of ~6.5 and smoother convergence than GRUScratch due to PyTorch's compiled operators. The output 'it has the time the time t' shows the model has learned meaningful words but still falls into repetition loops.

Config 1(more hidden) increased hidden units to 64 while keeping lr=4. This pushed train_ppl down to ~4 and maintained val_ppl at ~6.5, matching the base on validation but with better training performance and a faster, more stable convergence curve. The output 'it has is the filby the fi' shows improvement though coherence is still limited by the 50 epoch ceiling.

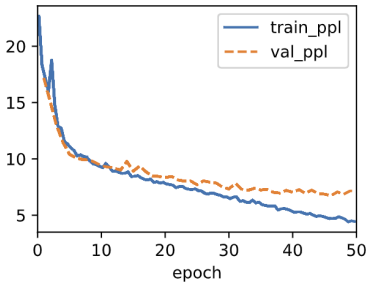
Config 2(lower lr) dropped the learning rate to 1 while keeping 64 hidden units. This was the worst performing configuration, both train and val perplexity stalled around ~8-8.5 and the output regressed to repetitive 'the the the'. The lower learning rate caused the model to converge too slowly to make meaningful progress in 50 epochs, demonstrating that lr=4 is well suited for this dataset.

Config 3(hidden & epochs) with 128 hidden units and 100 epochs produced the most interesting result: train_ppl dropped dramatically to ~2, but val_ppl exploded to ~15 by the end, a textbook case of severe overfitting. The model memorized the training sequences almost perfectly, as evidenced by the coherent output 'it has it always been rega', but completely failed to generalize. The val_ppl curve visibly diverges upward after epoch 40, showing exactly when overfitting takes over.

The best balanced configuration was Config 1: 64 hidden units, lr=4, 50 epochs, which achieved the strongest val_ppl without overfitting. The GRU's gating mechanism provides a clear advantage over vanilla RNNs, with the base GRU already outperforming the best vanilla RNN result from the previous project.

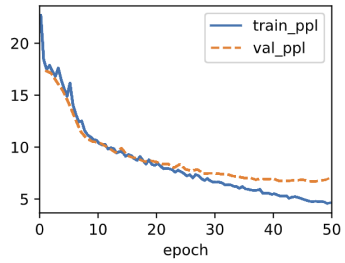
Task 2:

Graphs:



Reset Gate

Output:
'it has in the peaster a ma'



Update Gate

Output:
'it has i whing said filby '

Results:

Config	**train_ppl**	**val_ppl**	**Output**
Full GRU	~5	~6.5	‘the time the time’
Reset Gate	~4	~7	‘in the peaster a ma’
Update Gate	~4	~6.5	‘i whing said filby ‘

Conclusion:
The reset gate only variant removes the update gate by always fully replacing the hidden state at every time step. This means the model has no mechanism to decide how much of the previous hidden state to carry forward, it simply overwrites it entirely each step. The result is a train_ppl of ~4 but a val_ppl of ~7, which is the worst generalization of the three. The output 'it has in the peaster a ma' generates real looking words but lacks coherence, reflecting the

model's poor long term memory. Without the update gate, the model cannot preserve context across long sequences, which is exactly what the update gate is designed to do.

The update gate only variant removes the reset gate by fixing R=1, meaning the full previous hidden state always bleeds into the candidate computation without any selective forgetting. This variant actually performed surprisingly well matching the full GRU's val_ppl of ~6.5 and producing the most linguistically interesting output 'it has i whing said filby '. The convergence curve was also the smoothest of the two ablations. This suggests that on a relatively small dataset like Time Machine, the update gate contributes more to overall performance than the reset gate, as its ability to balance old and new information is the more critical operation.

Comparing both ablations to the full GRU, neither single gate variant matches the full model's combination of smooth convergence and generalization. The full GRU benefits from both gates working together, the reset gate handles short term context selectivity while the update gate manages long term memory retention. Removing either gate degrades performance, but removing the update gate is more damaging, confirming that the update gate is the more impactful of the two components in this setting.